AI Assistant Integration Guide

Overview

Spec files are designed to be AI-friendly. They live in Git, use structured markdown, and contain all the context an AI coding assistant needs to implement a feature correctly.

This guide shows how to leverage specs with AI assistants like GitHub Copilot, Cursor, Claude Code, and others.

Why Specs Work Well with AI Assistants

AI assistants work best when they have:

1. **Complete context** - specs consolidate business requirements, design decisions, and technical approach
2. **Structured format** - markdown sections are easy to parse and reference
3. **Version control** - specs live in Git alongside the code they describe
4. **Clear acceptance criteria** - AI can validate implementations against testable criteria

Specs address common AI coding problems:

- "I don't know what business logic to implement" → Context & Background section
- "I don't know what the UI should look like" → User Experience section
- "I don't know which architectural approach to use" → Technical Approach section
- "I don't know how to test this" → Testing Strategy section

How to Use Specs with AI Assistants

1. Context Loading (Before Starting)

Give the AI the full spec as context:

```
I'm implementing the feature described in [specs/EXAMPLE-2026-03-csv-
export.md](../specs/EXAMPLE-2026-03-csv-export.md).
Please read this spec and let me know when you're ready to start.
```

Why this works: AI reads the entire spec and understands the full scope before writing any code.

2. Section-by-Section Implementation

Reference specific sections when asking for code:

```
Based on the "Technical Approach" section of the spec, implement the
ExportButton component described in [specs/EXAMPLE-2026-03-csv-export.md]
```

```
(../specs/EXAMPLE-2026-03-csv-export.md).
```

Why this works: AI focuses on the relevant section, reducing hallucination and off-spec implementations.

3. Acceptance Criteria Validation

Ask AI to validate against acceptance criteria:

```
Review my implementation of the export feature against the "Acceptance
Criteria"
section in [specs/EXAMPLE-2026-03-csv-export.md](../specs/EXAMPLE-2026-03-
csv-export.md). Are there any criteria I haven't met?
```

Why this works: AI acts as a checklist, ensuring nothing is missed.

4. Test Generation

Use the Testing Strategy section for test guidance:

```
Generate unit tests for ExportButton.tsx based on the "Testing Strategy"
section
in [specs/EXAMPLE-2026-03-csv-export.md](../specs/EXAMPLE-2026-03-csv-
export.md).
```

Why this works: AI generates tests that match the spec's testing approach, not generic tests.

Best Practices

✅ DO

- **Load the spec first**: Give AI the full spec before asking for implementations
- **Reference sections explicitly**: "Based on the Technical Approach section..."
- **Validate against acceptance criteria**: Ask AI to check completeness
- **Use spec for test generation**: Reference Testing Strategy section
- **Update spec if requirements change**: Keep spec and code in sync

❌ DON'T

- **Don't ask AI to guess requirements**: Always provide the spec
- **Don't let AI deviate from the spec**: If AI suggests changes, discuss with team first
- **Don't skip validation**: Always check AI code against acceptance criteria
- **Don't assume AI remembers**: Re-load spec context if AI seems off-track

---

Example Workflows

Workflow 1: Implement a New Feature from Spec

```
Step 1: Load context
"I'm implementing the feature in [specs/EXAMPLE-2026-03-csv-export.md]
(../specs/EXAMPLE-2026-03-csv-export.md). Read the spec."

Step 2: Plan implementation
"Based on the spec, what order should I implement the components in?"

Step 3: Implement component by component
"Implement ExportButton.tsx according to the User Experience and Technical
Approach sections."

Step 4: Generate tests
"Generate unit tests based on the Testing Strategy section."

Step 5: Validate
"Review my implementation against all Acceptance Criteria. What's
missing?"
```

Workflow 2: Review Existing Code Against Spec

```
Step 1: Load spec and code
"Here's the spec ([specs/EXAMPLE-2026-03-csv-export.md](../specs/EXAMPLE-
2026-03-csv-export.md)) and my implementation
(src/components/ExportButton.tsx)."

Step 2: Ask for comparison
"Does my implementation match the spec? Highlight any deviations."

Step 3: Identify gaps
"Which Acceptance Criteria are not met by my current implementation?"

Step 4: Fix deviations
"Update my code to match the Technical Approach section more closely."
```

Workflow 3: Generate Documentation from Spec

```
"Based on [specs/EXAMPLE-2026-03-csv-export.md](../specs/EXAMPLE-2026-03-
csv-export.md), generate:
1. API documentation for the /api/exports endpoint
2. README section explaining how to use the export feature
3. User-facing help text for the export modal"
```

Tool-Specific Tips

GitHub Copilot

- Add spec path as a comment in your code file:

```
// Implementation of specs/EXAMPLE-2026-03-csv-export.md - ExportButton component
```

- Copilot will use nearby file context, including the spec if it's in the same repo

Cursor

- Use Cursor's "Add to Context" feature to load the spec file
- Reference specific sections: "@specs/EXAMPLE-2026-03-csv-export.md#technical-approach"

Claude Code (CLI)

- Use `/read` command to load spec:

```
/read [specs/EXAMPLE-2026-03-csv-export.md](../specs/EXAMPLE-2026-03-csv-export.md)
```

- Claude Code can read multiple files, so you can load spec + existing code simultaneously

ChatGPT / Claude (Web)

- Copy/paste the spec markdown into the chat
- Use headings to reference sections: "Based on the ## Technical Approach section..."

Keeping Specs and Code in Sync

When Code Diverges from Spec

If you discover the spec is wrong during implementation:

1. Stop coding
2. Discuss with PM/Design/Engineering
3. Update the spec first
4. Then continue implementation

If you make a small deviation (e.g., slightly different variable name):

- OK to proceed if it doesn't affect behavior
- Document deviation in PR description

If you make a significant deviation (e.g., different architecture):

- Not OK - this bypasses the spec review process

- Update spec, get team re-approval, then implement

Updating Specs After Implementation

When a feature is complete:

1. Update spec status to "Implemented"
2. If any deviations occurred, note them in "Revision History"
3. Commit spec update with implementation PR

```
git add [specs/EXAMPLE-2026-03-csv-export.md](../specs/EXAMPLE-2026-03-
csv-export.md)
git commit -m "docs: mark export spec as implemented

Feature merged in PR #456. Minor deviation: used Bull queue instead of SQS
for async jobs (team decision in standup on 2026-03-20).

Epic: PROD-123"
```

Troubleshooting

AI is generating code that doesn't match the spec

Problem: AI suggests a different approach than what's in the Technical Approach section.

Solution:

```
"Please stick to the architecture described in the Technical Approach
section
of [specs/EXAMPLE-2026-03-csv-export.md](../specs/EXAMPLE-2026-03-csv-
export.md). Don't suggest alternative approaches."
```

AI is missing acceptance criteria

Problem: Implementation looks good but fails some acceptance criteria.

Solution:

```
"Review all checkboxes in the Acceptance Criteria section. Which ones does
my
current implementation not satisfy?"
```

AI can't find the spec file

Problem: AI says it can't access the file.

Solution: Copy/paste the spec content directly into the chat, or use your tool's file upload feature.

Benefits of Spec-Driven AI Coding

Speed: AI implements faster when it has complete context upfront **Quality**: AI code matches team decisions (architecture, patterns, testing approach) **Consistency**: Multiple AI sessions produce similar implementations (they all follow the same spec) **Validation**: AI can self-check against acceptance criteria **Onboarding**: New team members + AI can ramp up quickly by reading specs

---

Example Prompts

Starting a Feature

```
I'm implementing the CSV export feature. The spec is at [specs/EXAMPLE-
2026-03-csv-export.md](../specs/EXAMPLE-2026-03-csv-export.md).

Step 1: Read the spec and summarize the key requirements.
Step 2: Suggest an implementation order for the components.
Step 3: Implement ExportButton.tsx according to the spec.
```

Reviewing Code

```
Here's my implementation of ExportButton.tsx [paste code].

Compare it to the spec at [specs/EXAMPLE-2026-03-csv-export.md]
(../specs/EXAMPLE-2026-03-csv-export.md).
- Does it match the User Experience section?
- Does it satisfy the Acceptance Criteria?
- What's missing or incorrect?
```

Generating Tests

```
Generate unit tests for ExportButton.tsx based on:
1. The Testing Strategy section in [specs/EXAMPLE-2026-03-csv-export.md]
(../specs/EXAMPLE-2026-03-csv-export.md)
2. The example test shown in that section
3. Coverage for all Acceptance Criteria related to the button
```

Updating Documentation

```
The export feature is now implemented. Based on [specs/EXAMPLE-2026-03-
csv-export.md](../specs/EXAMPLE-2026-03-csv-export.md),
```

```
generate user-facing documentation explaining:
- What the export feature does
- How to use it (step-by-step)
- What file formats are supported
- What to do if export fails
```

Questions?

This guide will evolve as the team gains experience using specs with AI assistants. Share your tips and workflows!

Navigation

Getting Started: Team README · Spec Template · Example Spec

Guides: Status Tracking · AI Integration · Pilot Discussion

Framework: Design Document · CLAUDE.md